

Software Testing Fundamentals

- Software Testing is an activity in software development.
- It is an investigation performed against a software to provide information about the quality of the software to stakeholders.

What is Software Testing?

- Different people have come up with various definitions for Software Testing, but generally, the aim is:
- To ensure that the software meets the agreed requirements and design
- The application works as expected
- The application doesn't contain serious bugs
- Meets its intended use as per user expectations
- Software testing is often used in association with the terms **verification** and **validation**.

White Box Testing

White Box Testing:

- White box testing is a testing technique that examines the program structure and derives test data from the program logic/code.
- The other names of glass box testing are clear box testing, open box testing, logic driven testing or path driven testing or structural testing.

White Box Testing Techniques:

- **Statement Coverage** - This technique is aimed at exercising all programming statements with minimal tests.
- **Branch Coverage** - This technique is running a series of tests to ensure that all branches are tested at least once.
- **Path Coverage** - This technique corresponds to testing all possible paths which means that each statement and branch is covered.

White Box Testing

Advantages of White Box Testing:

- Forces test developer to reason carefully about implementation.
- Reveals errors in "hidden" code.
- Spots the Dead Code or other issues with respect to best programming practices.

Disadvantages of White Box Testing:

- Expensive as one has to spend both time and money to perform white box testing.
- Every possibility that few lines of code are missed accidentally.
- In-depth knowledge about the programming language is necessary to perform white box testing.

Black-Box Testing

Black-Box Testing:

- The technique of testing without having any knowledge of the interior workings of the application is called black-box testing.
- The tester is oblivious to the system architecture and does not have access to the source code.
- Typically, while performing a black-box test, a tester will interact with the system's user interface by providing inputs and examining outputs without knowing how and where the inputs are worked upon.
- Black-box testing is a method of software testing that examines the functionality of an application based on the specifications.
- It is also known as Specifications based testing. Independent Testing Team usually performs this type of testing during the software testing life cycle.
- This method of test can be applied to each and every level of software testing such as unit, integration, system and acceptance testing.

Black-Box Testing

- **Behavioral Testing Techniques:**
- There are different techniques involved in Black Box testing.
- Equivalence Class
- Boundary Value Analysis
- Domain Tests
- Orthogonal Arrays
- Decision Tables
- State Models
- Exploratory Testing
- All-pairs testing

Black-Box Testing

Advantages:

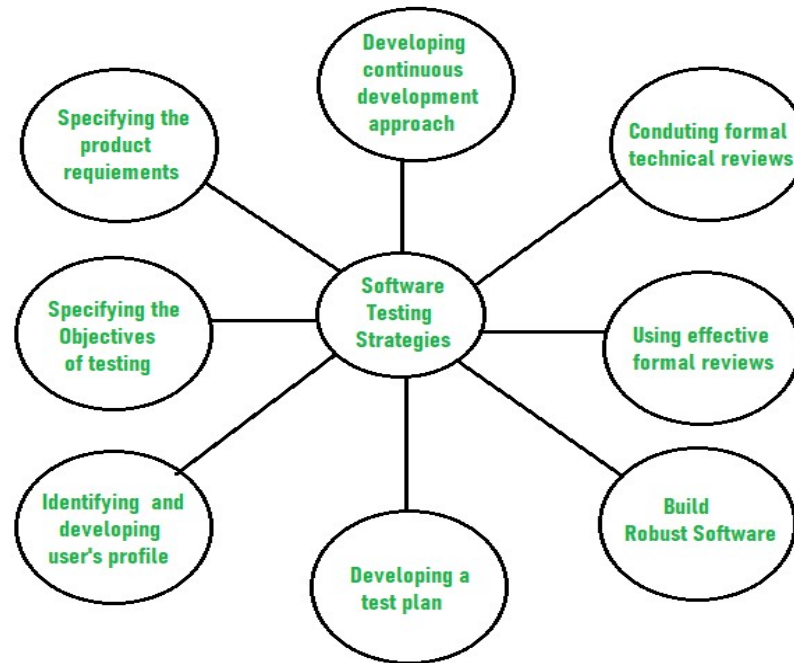
- Well suited and efficient for large code segments.
- Code access is not required.
- Clearly separates user's perspective from the developer's perspective through visibly defined roles.
- Large numbers of moderately skilled testers can test the application with no knowledge of implementation, programming language, or operating systems.

Disadvantages:

- Limited coverage, since only a selected number of test scenarios is actually performed.
- Inefficient testing, due to the fact that the tester only has limited knowledge about an application
- Blind coverage, since the tester cannot target specific code segments or error prone areas.
- The test cases are difficult to design.

Software Testing Strategies

- Software Testing is a type of investigation to find out if there is any default or error present in the software so that the errors can be reduced or removed to increase the quality of the software and to check whether it fulfills the specifies requirements or not.
- The overall strategy for testing software includes:



Software Testing Strategies

- 1. Before testing starts, it's necessary to identify and specify the requirements of the product in a quantifiable manner.**
 - Different characteristics quality of the software is there such as maintainability that means the ability to update and modify, the probability that means to find and estimate any risk, and usability that means how it can easily be used by the customers or end-users.
 - All these characteristic qualities should be specified in a particular order to obtain clear test results without any error.
- 2. Specifying the objectives of testing in a clear and detailed manner.**
 - Several objectives of testing are there such as effectiveness that means how effectively the software can achieve the target, any failure that means inability to fulfill the requirements and perform functions, and the cost of defects or errors that mean the cost required to fix the error.
 - All these objectives should be clearly mentioned in the test plan.

Software Testing Strategies

- 3. For the software, identifying the user's category and developing a profile for each user.**
 - Use cases describe the interactions and communication among different classes of users and the system to achieve the target.
 - So as to identify the actual requirement of the users and then testing the actual use of the product.
- 4. Developing a test plan to give value and focus on rapid-cycle testing.**
 - Rapid Cycle Testing is a type of test that improves quality by identifying and measuring the any changes that need to be required for improving the process of software.
 - Therefore, a test plan is an important and effective document that helps the tester to perform rapid cycle testing.

Software Testing Strategies

5. Robust software is developed that is designed to test itself.

- The software should be capable of detecting or identifying different classes of errors
- Moreover, software design should allow automated and regression testing which tests the software to find out if there is any adverse or side effect on the features of software due to any change in code or program.

6. Before testing, using effective formal reviews as a filter.

- Formal technical reviews is technique to identify the errors that are not discovered yet.
- The effective technical reviews conducted before testing reduces a significant amount of testing efforts and time duration required for testing software so that the overall development time of software is reduced.

Software Testing Strategies

7. Conduct formal technical reviews to evaluate the nature, quality or ability of the test strategy and test cases.

- The formal technical review helps in detecting any unfilled gap in the testing approach.
- Hence, it is necessary to evaluate the ability and quality of the test strategy and test cases by technical reviewers to improve the quality of software.

8. For the testing process, developing a approach for the continuous development.

- As a part of a statistical process control approach, a test strategy that is already measured should be used for software testing to measure and control the quality during the development of software.

Verification and Validation

- Verification and Validation is the process of investigating that a software system satisfies specifications and standards and it fulfills the required purpose.
- **Barry Boehm** described verification and validation as the following:
- ***Verification:*** *Are we building the product right?*
Validation: *Are we building the right product?*

Verification:

- Verification is the process of checking that a software achieves its goal without any bugs.
- It is the process to ensure whether the product that is developed is right or not.
- It verifies whether the developed product fulfills the requirements that we have.
Verification is **Static Testing**.

Activities involved in verification:

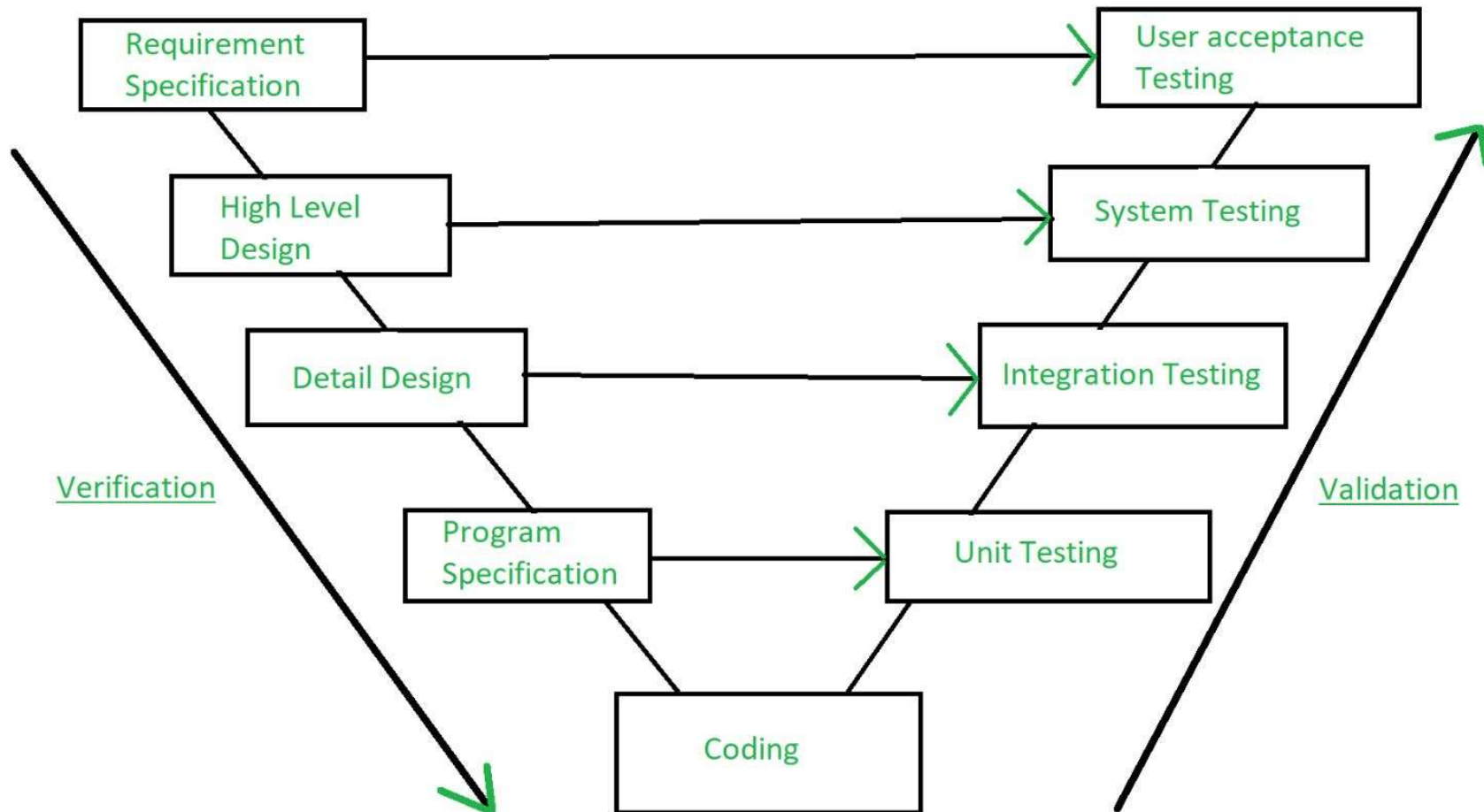
- Inspections
- Reviews
- Walkthroughs
- Desk-checking

Verification and Validation

Validation:

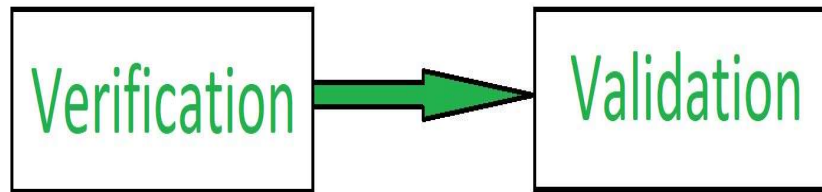
- Validation is the process of checking whether the software product is up to the mark or in other words product has high level requirements.
- It is the process of checking the validation of product i.e. it checks what we are developing is the right product. it is validation of actual and expected product.
- Validation is the **Dynamic Testing**.
- **Activities involved in validation:**
 - Black box testing
 - White box testing
 - Unit testing
 - Integration testing

Verification and Validation



Verification and Validation

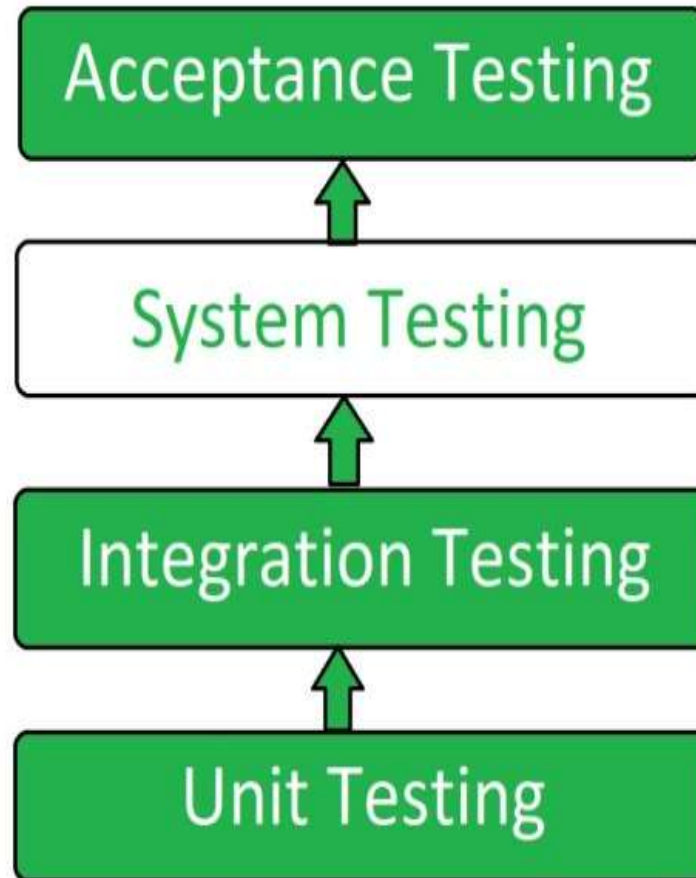
Note: Verification is followed by Validation.



System Testing

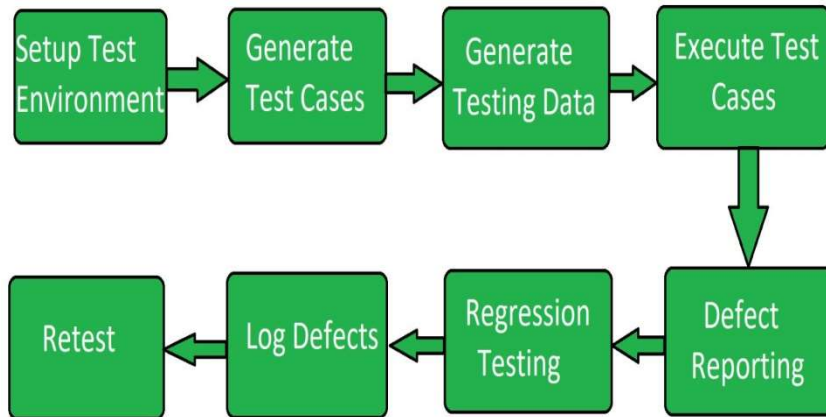
- **System Testing** is a type of software testing that is performed on a complete integrated system to evaluate the compliance of the system with the corresponding requirements.
- In system testing, integration testing passed components are taken as input.
- The goal of integration testing is to detect any irregularity between the units that are integrated together.
- System testing detects defects within both the integrated units and the whole system. The result of system testing is the observed behavior of a component or a system when it is tested.
- **System Testing** is carried out on the whole system in the context of either system requirement specifications or functional requirement specifications or in the context of both.
- System testing tests the design and behavior of the system and also the expectations of the customer.
- It is performed to test the system beyond the bounds mentioned in the software requirements specification (SRS).
- System Testing is basically performed by a testing team that is independent of the development team that helps to test the quality of the system impartial.
- It has both functional and non-functional testing.
- **System Testing is a black-box testing.**
- System Testing is performed after the integration testing and before the acceptance testing.

System Testing



System Testing

System Testing is performed in the following steps:



1. Test Environment Setup:

Create testing environment for the better quality testing.

2. Create Test Case:

Generate test case for the testing process.

3. Create Test Data:

Generate the data that is to be tested.

4. Execute Test Case:

After the generation of the test case and the test data, test cases are executed.

5. Defect Reporting:

Defects in the system are detected.

6. Regression Testing:

It is carried out to test the side effects of the testing process.

7. Log Defects:

Defects are fixed in this step.

8. Retest:

If the test is not successful then again test is performed.

System Testing

Types of System Testing:

1. Performance Testing:

Performance Testing is a type of software testing that is carried out to test the speed, scalability, stability and reliability of the software product or application.

2. Load Testing:

Load Testing is a type of software Testing which is carried out to determine the behavior of a system or software product under extreme load.

3. Stress Testing:

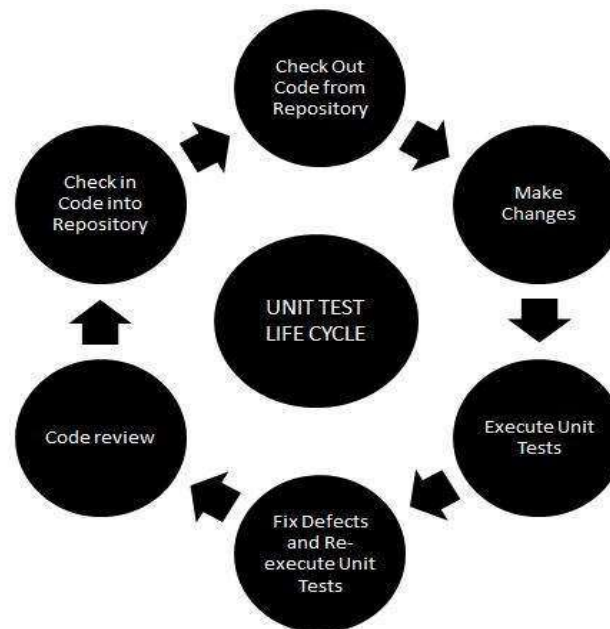
Stress Testing is a type of software testing performed to check the robustness of the system under the varying loads.

4. Scalability Testing:

Scalability Testing is a type of software testing which is carried out to check the performance of a software application or system in terms of its capability to scale up or scale down the number of user request load.

Unit Testing

- It is a testing technique using which individual modules are tested to determine if there are any issues by the developer himself.
- It is concerned with functional correctness of the standalone modules.
- The main aim is to isolate each unit of the system to identify, analyze and fix the defects.
- **Unit Testing LifeCycle:**



Unit Testing

Unit Testing Techniques:

- **Black Box Testing** - Using which the user interface, input and output are tested.
- **White Box Testing** - used to test each one of those functions behavior is tested.
- **Gray Box Testing** - Used to execute tests, risks and assessment methods.

Advantages:

- Reduces Defects in the Newly developed features or reduces bugs when changing the existing functionality.
- Reduces Cost of Testing as defects are captured in very early phase.
- Improves design and allows better refactoring of code.
- Unit Tests, when integrated with build gives the quality of the build as well.

Integration testing

- **Integration testing** is the process of testing the interface between two software units or module.
- It's focus on determining the correctness of the interface.
- The purpose of the integration testing is to expose faults in the interaction between integrated units.
- Once all the modules have been unit tested, integration testing is performed.

Integration testing

Integration test approaches –

There are four types of integration testing approaches. Those approaches are the following:

1. Big-Bang Integration Testing –

- It is the simplest integration testing approach, where all the modules are combining and verifying the functionality after the completion of individual module testing.
- In simple words, all the modules of the system are simply put together and tested.
- This approach is practicable only for very small systems.
- If once an error is found during the integration testing, it is very difficult to localize the error as the error may potentially belong to any of the modules being integrated.
- So, debugging errors reported during big bang integration testing are very expensive to fix.

Advantages:

- It is convenient for small systems.

Disadvantages:

- There will be quite a lot of delay because you would have to wait for all the modules to be integrated.
- High risk critical modules are not isolated and tested on priority since all modules are tested at once.

Integration testing

2. Bottom-Up Integration Testing –

- In bottom-up testing, each module at lower levels is tested with higher modules until all modules are tested.
- The primary purpose of this integration testing is, each subsystem is to test the interfaces among various modules making up the subsystem.
- This integration testing uses test drivers to drive and pass appropriate data to the lower level modules.

Advantages:

- In bottom-up testing, no stubs are required.
- A principle advantage of this integration testing is that several disjoint subsystems can be tested simultaneously.
- **Disadvantages:**
- Driver modules must be produced.
- In this testing, the complexity that occurs when the system is made up of a large number of small subsystem.

Integration testing

3. Top-Down Integration Testing –

- Top-down integration testing technique used in order to simulate the behavior of the lower-level modules that are not yet integrated.
- In this integration testing, testing takes place from top to bottom.
- First high-level modules are tested and then low-level modules and finally integrating the low-level modules to a high level to ensure the system is working as intended.

Advantages:

- Separately debugged module.
- Few or no drivers needed.
- It is more stable and accurate at the aggregate level.

Disadvantages:

- Needs many Stubs.
- Modules at lower level are tested inadequately.

Integration testing

4. Mixed Integration Testing –

- A mixed integration testing is also called sandwiched integration testing.
- A mixed integration testing follows a combination of top down and bottom-up testing approaches.
- In top-down approach, testing can start only after the top-level module have been coded and unit tested.
- In bottom-up approach, testing can start only after the bottom level modules are ready.
- This sandwich or mixed approach overcomes this shortcoming of the top-down and bottom-up approaches.
- A mixed integration testing is also called sandwiched integration testing.

Advantages:

- Mixed approach is useful for very large projects having several sub projects.
- This Sandwich approach overcomes this shortcoming of the top-down and bottom-up approaches.

Disadvantages:

- For mixed integration testing, require very high cost because one part has Top-down approach while another part has bottom-up approach.
- This integration testing cannot be used for smaller system with huge interdependence between different modules.

Debugging

- Debugging is the process of fixing a bug in the software.
- In other words, it refers to identifying, analyzing and removing errors.
- This activity begins after the software fails to execute properly and concludes by solving the problem and successfully testing the software.
- It is considered to be an extremely complex and tedious task because errors need to be resolved at all stages of debugging.

Debugging Process: Steps involved in debugging are:

- Problem identification and report preparation.
- Assigning the report to software engineer to the defect to verify that it is genuine.
- Defect Analysis using modeling, documentations, finding and testing candidate flaws, etc.
- Defect Resolution by making required changes to the system.
- Validation of corrections.

Debugging

Debugging Strategies:

1. Study the system for the larger duration in order to understand the system. It helps debugger to construct different representations of systems to be debugging depends on the need. Study of the system is also done actively to find recent changes made to the software.
2. Backwards analysis of the problem which involves tracing the program backward from the location of failure message in order to identify the region of faulty code. A detailed study of the region is conducting to find the cause of defects.
3. Forward analysis of the program involves tracing the program forwards using breakpoints or print statements at different points in the program and studying the results. The region where the wrong outputs are obtained is the region that needs to be focused to find the defect.
4. Using the past experience of the software debug the software with similar problems in nature. The success of this approach depends on the expertise of the debugger.

Debugging

Debugging Tools:

- Debugging tool is a computer program that is used to test and debug other programs.
- A lot of public domain software like gdb and dbx are available for debugging.
- They offer console-based command line interfaces.
- Examples of automated debugging tools include code based tracers, profilers, interpreters, etc.
- Some of the widely used debuggers are: Radare2, WinDbg, Valgrind require very high cost because one part has Top-down approach while another part has bottom-up approach.
- This integration testing cannot be used for smaller system with huge interdependence between different modules.

Implementation Types

Implementation is the process of launching a change to system, process, policies, data, equipment, infrastructure, facilities and information.

Implementation Types:

1. Parallel: When the new system is used at the same time as the old system the two systems are said to be running in parallel.

Advantages:

- Users can compare the output of the old system with the output of the new system, to ensure correctness
- There is little risk of data loss because the known-good system is running

Disadvantages:

- Users must take more time to enter data into two different systems
- Data could be different in two different systems if there is intensive data entry.

Example:

- A medical system that tracks patient heart rates is being replaced. A new system is attached while the old system is still working. The two systems are used in parallel to ensure the new system produces the exact same data as the old system.

Implementation Types

2. Phased:

- When small parts of the new system gradually replace small parts of the old system, the implementation method is said to be phased.

Advantages:

- Training can be completed in small parts
- A failure of the new system has minimal impact because it is only one small part
- Issues around scale can be addressed without major impact.

Disadvantages:

- This implementation method takes more time to get the new system fully online than other methods.
- There is a possibility of data loss if part of the new system fails.

Example:

- A school has a new system to manage student athletics. The old system is paper and pencil. Slowly, over time, a new system is introduced to manage students, their teams, seasons, and their coaches. At first, the new system simply manages teams. Then the new system manages seasons (and school years), slowly, the new system is increased to manage coaches, players and finally events. At the end of implementation, the new system is managing everything related to student athletics and the old paper and pencil system isn't being used any longer.

Implementation Types

3. Pilot :

- When a small group of users within an organization uses a new system prior to wider use, the system is said to be piloted.

Advantages:

- Training can be supported by pilot group
- Failure or problems can be identified and addressed without wide-spread impact to the organization

Disadvantages:

- In a pilot, issues of scale can cause problems. For example, the system might work well for 10 users, but not for 1000.

Example:

- A bakery is implementing a new system for customers to order online. They choose 50 customers and ask them to try the new system, and provide feedback. The bakery can then identify issues and address them prior to implementing systems for thousands of users.

Implementation Types

4. Direct:

- When a new system is implemented without any phased or pilot implementation, it is said to be direct. The old system is retired, and the new system goes live.

Advantages:

- If the system is not critical, this can be a good method for implementation

Disadvantages:

- If you are not sure the system will work, this method of implementation may not be a good idea

Example:

- A store is implementing a new electronic system for employees to leave suggestions for improvement. There is no existing system. The store uses direct method because they are very sure the new system will work, there is a low cost if the system fails, and the store wants to make a "big splash" with the new system.

Software Maintenance

Software Maintenance:

- Software Maintenance is the process of modifying a software product after it has been delivered to the customer. The main purpose of software maintenance is to modify and update software application after delivery to correct faults and to improve performance.

Need for Maintenance –

Software Maintenance must be performed in order to:

- Correct faults.
- Improve the design.
- Implement enhancements.
- Interface with other systems.
- Accommodate programs so that different hardware, software, system features, and telecommunications facilities can be used.

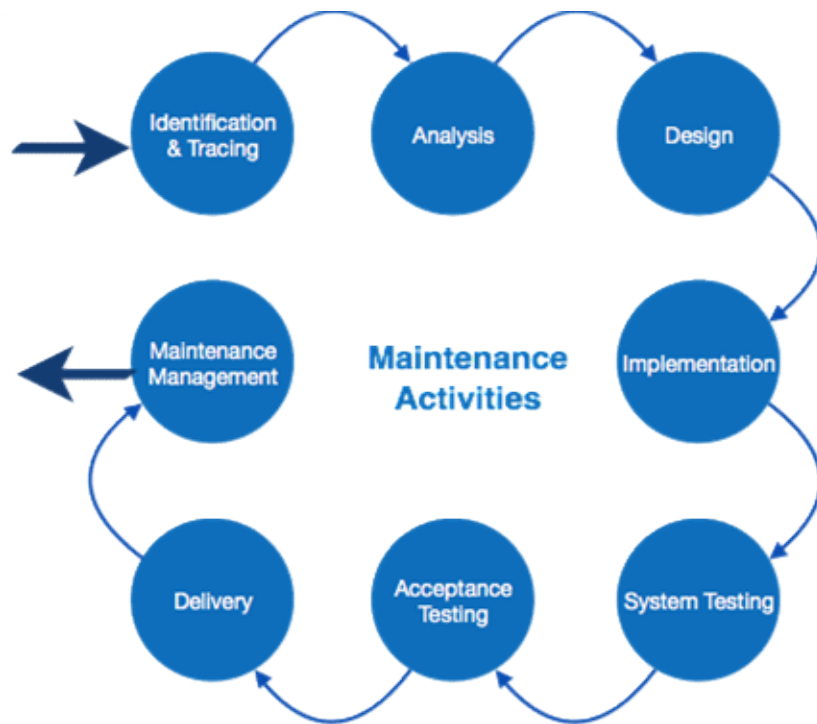
Software Maintenance

Categories of Software Maintenance –

Maintenance can be divided into the following:

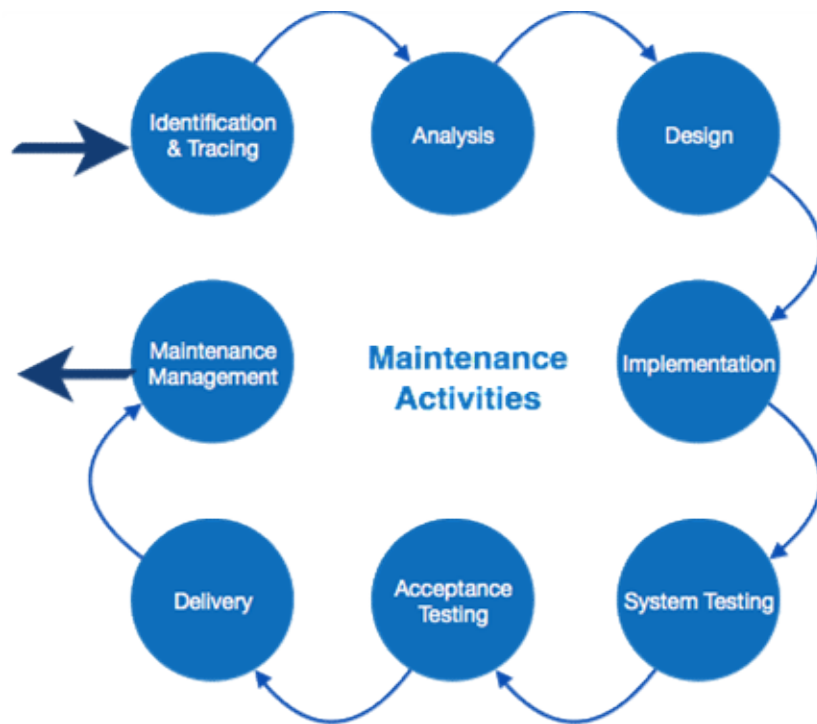
- **Corrective maintenance:**
Corrective maintenance of a software product may be essential either to rectify some bugs observed while the system is in use, or to enhance the performance of the system.
- **Adaptive maintenance:**
This includes modifications and updations when the customers need the product to run on new platforms, on new operating systems, or when they need the product to interface with new hardware and software.
- **Perfective maintenance:**
A software product needs maintenance to support the new features that the users want or to change different types of functionalities of the system according to the customer demands.
- **Preventive maintenance:**
This type of maintenance includes modifications and updations to prevent future problems of the software. It goals to attend problems, which are not significant at this moment but may cause serious issues in future.

Maintenance Activities



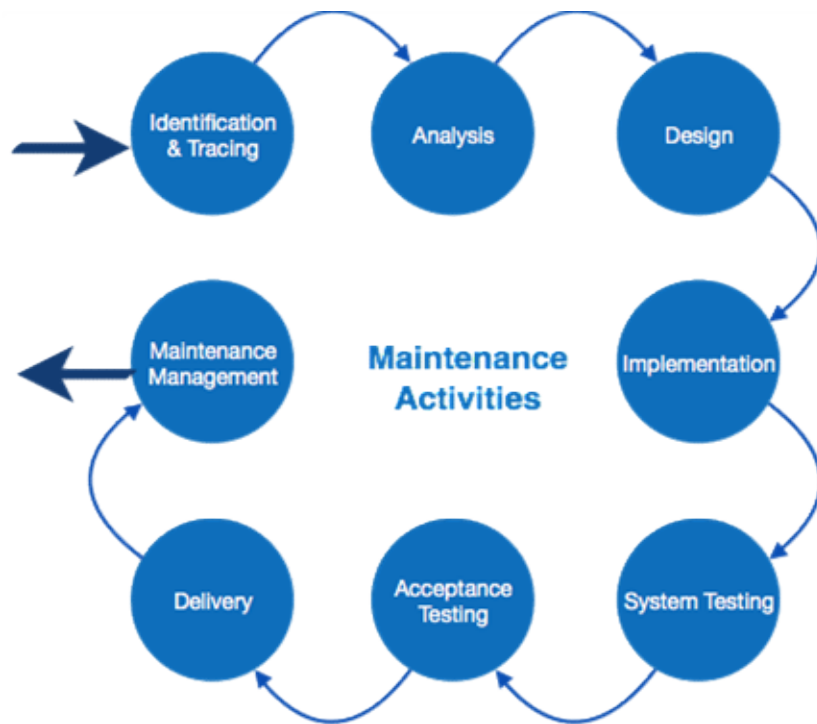
- IEEE provides a framework for sequential maintenance process activities.
- It can be used in iterative manner and can be extended so that customized items and processes can be included.
- These activities go hand-in-hand with each of the following phase:
- **Identification & Tracing** - It involves activities pertaining to identification of requirement of modification or maintenance. It is generated by user or system may itself report via logs or error messages. Here, the maintenance type is classified also.
- .

Maintenance Activities



- **Analysis** - The modification is analyzed for its impact on the system including safety and security implications. If probable impact is severe, alternative solution is looked for. A set of required modifications is then materialized into requirement specifications. The cost of modification/maintenance is analyzed and estimation is concluded
- **Design** - New modules, which need to be replaced or modified, are designed against requirement specifications set in the previous stage. Test cases are created for validation and verification.
- **Implementation** - The new modules are coded with the help of structured design created in the design step. Every programmer is expected to do unit testing in parallel.

Maintenance Activities



- **System Testing** - Integration testing is done among newly created modules. Integration testing is also carried out between new modules and the system. Finally the system is tested as a whole, following regressive testing procedures.
- **Acceptance Testing** - After testing the system internally, it is tested for acceptance with the help of users. If at this state, user complaints some issues they are addressed or noted to address in next iteration.
- **Delivery** - After acceptance test, the system is deployed all over the organization either by small update package or fresh installation of the system. The final testing takes place at client end after the software is delivered.